



Manual técnico — Galsoft Helpdesk

Documentación para evaluación académica: arquitectura, base de datos, módulos y despliegue.

Repositorio: GitHub, rama `main`

Carpeta local: la ruta depende de cada usuario y de dónde clone el repositorio. Lo importante es entrar en la carpeta donde está `app.py`.

Stack: Python 3 · Flask · MySQL · Bootstrap 5

URL local: <http://127.0.0.1:5000>

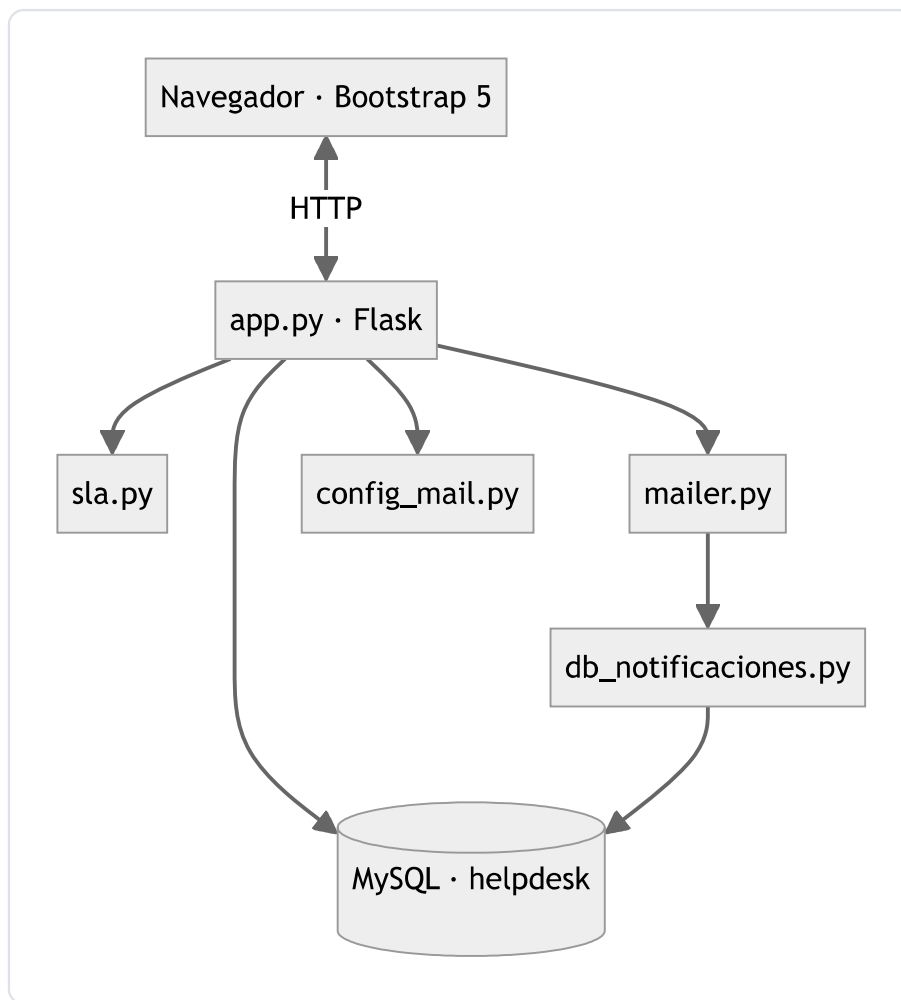
1. Objetivo del sistema

Aplicación web de mesa de ayuda (helpdesk) que permite:

- A los **clientes**: abrir peticiones, consultar estado y comentar.
- A los **agentes**: gestionar tickets, asignarse incidencias, responder y controlar SLA.
- A los **administradores**: gestionar usuarios, categorías y revisar el historial de correos.

La base de datos se entrega completa en `helpdesk.sql` (export de phpMyAdmin).

2. Arquitectura general



Flujo resumido: el navegador habla con Flask (`app.py`); la lógica auxiliar está en `sla.py` , `mailer.py` y `config_mail.py` ; los correos se registran con `db_notificaciones.py` ; todo persiste en MySQL.

- **Presentación:** plantillas Jinja2 en `templates/` + estilos en `static/css/styles.css` .
- **Lógica de negocio:** concentrada en `app.py` , con módulos auxiliares para SLA y correo.
- **Persistencia:** MySQL mediante `mysql.connector` , cursor global `micursor` .

3. Estructura de archivos

Ruta	Descripción
<code>app.py</code>	Aplicación Flask: rutas, sesión, consultas SQL, orquestación
<code>sla.py</code>	Cálculo de plazos SLA y evaluación (en plazo / riesgo /

Ruta	Descripción
	vencido)
<code>mailer.py</code>	Envío SMTP y avisos automáticos (ticket nuevo, comentarios, etc.)
<code>config_mail.py</code>	Carga <code>mail.env</code> y variables de entorno de correo
<code>db_notificaciones.py</code>	Insertión y listado de <code>notificaciones</code>
<code>mail.env</code>	Configuración SMTP real (no subir a repositorio; crear desde <code>mail.env.example</code>)
<code>mail.env.example</code>	Plantilla de configuración de correo
<code>helpdesk.sql</code>	Base de datos completa (estructura + datos) para importar en MySQL
<code>templates/</code>	Vistas HTML (base, index, detalle, dashboard, login, etc.)
<code>static/css/styles.css</code>	Estilos Galsoft (paleta pastel, sidebar, tarjetas)
<code>static/LogoGalsoft.png</code>	Logo corporativo
<code>MANUAL_USUARIO.md</code>	Manual de uso para usuarios finales
<code>MANUAL_TECNICO.md</code>	Este documento

4. Base de datos `helpdesk`

4.1 Instalación de la base de datos

1. Abrir **phpMyAdmin** (o consola MySQL).
2. Pestaña **Importar** → seleccionar `helpdesk.sql` .
3. Ejecutar. El script crea la base `helpdesk` , las tablas y los datos de ejemplo.

4.2 Tablas del sistema

usuarios

Columna	Tipo	Descripción
id	INT PK AI	Identificador
nombre	VARCHAR	Nombre completo
email	VARCHAR	Correo (login único)
password	VARCHAR	Hash bcrypt (werkzeug.security)
rol	VARCHAR	cliente , agente o admin

tickets

Columna	Tipo	Descripción
id	INT PK AI	Número de ticket
titulo	VARCHAR	Asunto breve
descripcion	TEXT	Detalle de la incidencia
estado	VARCHAR	Abierto , En proceso , Cerrado (por defecto Abierto)
prioridad	VARCHAR	baja , media , alta
archivo	VARCHAR	Nombre del fichero adjunto
archivo_blob	LOB	Contenido binario del adjunto
usuario_id	INT FK	Cliente que abrió el ticket
agente_id	INT FK	Agente asignado; NULL = sin asignar

Columna	Tipo	Descripción
categoria_id	INT FK	Categoría del ticket
cerrado_en	DATETIME	Fecha/hora de cierre
sla_respuesta_limite	DATETIME	Plazo máximo de primera respuesta
sla_resolucion_limite	DATETIME	Plazo máximo de resolución
primera_respuesta_en	DATETIME	Primera respuesta pública de agente/admin
creado_en	DATETIME	Fecha de alta

categorias

Columna	Tipo	Descripción
id	INT PK AI	Identificador
nombre	VARCHAR	Nombre único (General, Hardware, Software, etc.)

ticket_historial

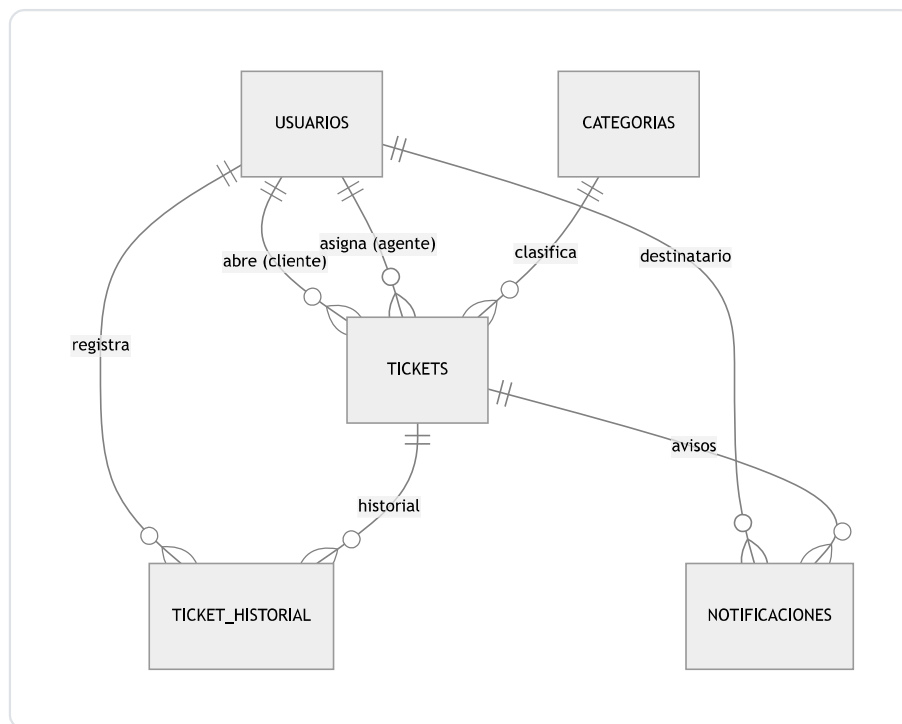
Columna	Tipo	Descripción
id	INT PK AI	Identificador
ticket_id	INT FK	Ticket relacionado
usuario_id	INT FK	Quién realizó la acción
tipo	VARCHAR	creado , comentario , estado , prioridad , asignacion , archivo , etc.
detalle	TEXT	Texto descriptivo

Columna	Tipo	Descripción
<code>es_interno</code>	TINYINT	1 = nota solo visible para agente/admin
<code>fecha</code>	DATETIME	Marca temporal

notificaciones

Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Identificador
<code>ticket_id</code>	INT FK	Ticket relacionado
<code>email</code>	VARCHAR	Destinatario
<code>tipo</code>	VARCHAR	Evento (<code>ticket_creado_cliente</code> , <code>comentario_agente</code> , etc.)
<code>asunto</code>	VARCHAR	Asunto del correo
<code>cuerpo</code>	TEXT	Cuerpo del mensaje
<code>enviado</code>	TINYINT	1 = enviado correctamente
<code>error_msg</code>	VARCHAR	Error SMTP si falló
<code>creado_en</code>	DATETIME	Fecha del intento de envío

4.3 Diagrama entidad-relación (simplificado)



5. Rutas Flask (`app.py`)

Cada ruta lleva un comentario `# RUTA:` en el código fuente.

Ruta	Función	Acceso
<code>/</code>	Listado de tickets con filtros	Sesión requerida
<code>/ticket/nuevo</code>	Crear ticket	Cliente+
<code>/login</code>	Autenticación	Público
<code>/register</code>	Alta de cliente	Público
<code>/logout</code>	Cerrar sesión	Sesión
<code>/descargar/<id></code>	Descarga adjunto BLOB	Según permisos
<code>/ticket/<id></code>	Detalle, comentarios, gestión	Según rol
<code>/usuarios</code>	Listado usuarios	Admin
<code>/usuario/editar/<id></code>	Editar usuario	Admin
<code>/categorias</code>	CRUD categorías	Admin

Ruta	Función	Acceso
<code>/dashboard</code>	Estadísticas y SLA	Agente, admin
<code>/notificaciones</code>	Historial de correos	Admin
<code>/manual/usuario</code>	Manual de usuario (HTML)	Público
<code>/manual/tecnico</code>	Manual técnico (HTML)	Público

Sesión Flask (`session`): `usuario_id`, `usuario_nombre`, `usuario_rol`.

Clave secreta: `app.secret_key = os.urandom(24)` — las sesiones se invalidan al reiniciar el servidor (comportamiento documentado en FAQ).

6. Módulos auxiliares

6.1 `sla.py`

- Constante `SLA_HORAS` : plazos por prioridad (baja/media/alta).
- `calcular_limites_sla(prioridad, creado_en)` → tupla (límite respuesta, límite resolución).
- `evaluar_sla(ticket)` → diccionario con estados `ok`, `riesgo`, `vencido`, `cumplido`.
- `RIESGO_FRACCION = 0.25` : último 25 % del plazo = «en riesgo».
- `marcar_primera_respuesta()` al primer comentario público de agente/admin.

6.2 `mailer.py` + `config_mail.py`

- Lee `mail.env` (IONOS, Gmail u otro SMTP).
- `MAIL_ENABLED=0` desactiva envío; la app sigue funcionando.
- Eventos que disparan correo: ticket nuevo, comentarios, cambio estado, asignación.

- Cada intento se registra en `notificaciones` vía `db_notificaciones.py`.

6.3 `db_notificaciones.py`

- Detecta columnas reales de `notificaciones` (`creado_en`, `fecha_envio`, `mensaje`, etc.).
- Normaliza filas para plantillas aunque la tabla tenga columnas de esquemas anteriores mezcladas.

7. Seguridad y buenas prácticas implementadas

Aspecto	Implementación
Contraseñas	<code>generate_password_hash</code> / <code>check_password_hash</code> (Werkzeug)
SQL injection	Consultas parametrizadas con <code>%s</code> y tuplas de valores
Archivos	<code>secure_filename</code> en subida; almacenamiento en BLOB
Autorización	Comprobación de <code>session</code> y <code>usuario_rol</code> en cada ruta sensible
Notas internas	Filtro <code>es_interno = 0</code> en historial para clientes
Tickets cliente	<code>WHERE t.usuario_id = %s</code> en consultas de detalle/listado

Pendiente para producción: HTTPS, variables de entorno para BD, `secret_key` fija, rate limiting, CSRF en formularios.

8. Interfaz de usuario

- **Bootstrap 5.3.3** (CDN).
- **Sidebar** fijo bajo navbar (`top: 64px`); toggle móvil con ☰.
- **Listado en tarjetas** con badges de estado, prioridad y SLA.
- **Filtro Jinja** `fecha_es` : formato `DD-MM-AAAA HH:MM`.

- **Paleta:** verde (ok/abierto/baja), amarillo (proceso/media/riesgo), rojo (cerrado/alta/vencido).

Plantillas principales: `base.html` , `index.html` ,
`ticket_detalle.html` , `dashboard.html` , `login.html` ,
`register.html` , `new_ticket.html` , `usuarios.html` ,
`editar_usuario.html` , `categorias.html` ,
`notificaciones.html` , `manual_view.html` .

9. Instalación y ejecución

9.1 Obtener el proyecto

1. Clonar el repositorio desde GitHub (rama `main`). GitHub Desktop o `git clone` crearán la carpeta en la ruta que elija cada usuario.
2. Abrir esa carpeta: es la raíz del proyecto (contiene `app.py` , `helpdesk.sql` y `templates/`).

9.2 Requisitos

- Python 3.x
- MySQL / MariaDB (servicio activo)
- Paquetes: `flask` , `mysql-connector-python` , `werkzeug`

```
pip install flask mysql-connector-python werkzeug
```

9.3 Base de datos

1. Importar `helpdesk.sql` en phpMyAdmin (crea la base, tablas y datos).
2. Verificar conexión en `app.py` :

```
mibd = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="",  
    database="helpdesk"  
)
```

9.4 Correo (opcional)

Copiar la plantilla y editarla (Windows: `copy` ; Linux/Mac: `cp`):

```
copy mail.env.example mail.env
```

Editar `mail.env` con datos SMTP (ejemplo IONOS:
`smtp.ionos.es` , puerto 587, STARTTLS).

9.5 Arranque

Desde la **raíz del repositorio clonado** (carpeta con `app.py`):

```
python app.py
```

Abrir <http://127.0.0.1:5000>

10. Flujos de negocio relevantes

10.1 Creación de ticket

1. POST `/ticket/nuevo` → INSERT en `tickets` con límites SLA.
2. `registrar_historial(..., 'creado', ...)` .
3. `mailer.on_ticket_nuevo()` → cliente y equipo.

10.2 Tomar ticket (agente)

1. UPDATE `agente_id` , posible cambio a «En proceso».
2. Historial tipo `asignacion` .
3. Notificación al agente si aplica.

10.3 Comentario agente (público)

1. INSERT historial `comentario` .
2. `marcar_primera_respuesta` si es la primera.
3. `mailer` avisa al cliente.

10.4 Cierre

1. Estado `Cerrado` , `cerrado_en = NOW()` .
 2. Historial + correo al cliente.
-

11. Consultas SQL destacadas

- **Listado base:** `SQL_TICKET_BASE` — JOIN cliente, agente y categoría.
 - **Filtro SLA vencido:** compara `NOW()` con `sla_respuesta_limite / sla_resolucion_limite`.
 - **Dashboard:** agregaciones `COUNT`, `GROUP BY` por agente y categoría.
 - **Historial:** orden `fecha DESC`, filtro `es_interno` para clientes.
-

12. Limitaciones conocidas

- Conexión MySQL única global (no pool); adecuado para demo/local.
 - `secret_key` aleatoria reinicia sesiones al reiniciar Flask.
 - Adjuntos en BLOB (no escalable para ficheros muy grandes).
 - Sin API REST ni websockets; interfaz 100 % servidor-renderizada.
 - Correo depende de credenciales SMTP externas (`mail.env`).
-

13. Referencias para evaluación

Documento	Audiencia
<code>MANUAL_USUARIO.md</code>	Usuarios finales (cliente, agente, admin)
<code>MANUAL_TECNICO.md</code>	Revisión técnica
<code>helpdesk.sql</code>	Volcado completo de la base de datos
Comentarios <code># RUTA:</code> y <code># FUNCIÓN:</code> en <code>app.py</code>	Mapa rápido del código

Ver en la aplicación: <http://127.0.0.1:5000/manual/tecnico>
(también desde el pie de página).
